

Project Based Learning Report
ON
Page Pilot AI web page summariser

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY
CST - IIIrd Year
Section - D
Ayush Sharma (00418012723)
Sumit kr. pandit (01318012723)
Harsh (01218012723)
Rajguru (02818012723)

GUIDED BY
Ms. Gunjan Sethi
(Assistant Professor)

Department of Computer Science & Engineering
DELHI TECHNICAL CAMPUS, GREATER NOIDA
(Affiliated by Guru Gobind Singh Indraprastha University, New Delhi) Session 2025-26
(Even Sem)

TABLE OF CONTENTS

- 1. ABSTRACT**
- 2. INTRODUCTION**
- 3. PROBLEM STATEMENT**
- 4. OBJECTIVE**
- 5. METHODOLOGY**
- 6. SOURCE CODE SNIPPETS**
- 7. PURPOSE OF WORK**
- 8. FUTURE SCOPE**
- 9. CONCLUSION**
- 10. REFERENCES**

ABSTRACT:

PagePilot AI is a smart, lightweight browser extension built to combat the growing problem of information overload on the modern web. Designed with a performance-first approach, it leverages a robust tech stack including **React.js**, **Node.js**, and the **Google Gemini API** to deliver instant, context-aware summaries of long-form content such as articles, research papers, and job listings.

The extension intelligently extracts and processes webpage content in real time, converting dense information into clear, structured bullet points—helping users grasp key insights within seconds. To handle large-scale and complex data efficiently, PagePilot AI integrates **LangChain**, enabling advanced text chunking, semantic understanding, and context preservation across long documents.

Beyond summarization, PagePilot AI offers a **conversational AI interface**, allowing users to ask follow-up questions about the content directly within the page. This transforms passive reading into an interactive learning experience, where users can dive deeper without leaving the context of the original source.

The platform is designed with **accessibility and personalization** at its core. It supports **multi-lingual outputs**, including Hinglish, making information more inclusive and easier to consume for diverse user groups. Users can also customize summary depth, tone (formal, casual), and format based on their preferences.

To ensure seamless performance, PagePilot AI employs **Redis caching** for faster response times and reduced API calls, along with a **Manifest V3 architecture** for enhanced security, efficiency, and compliance with modern browser standards. The extension is optimized for minimal resource usage, ensuring it runs smoothly without impacting browsing speed.

INTRODUCTION:

In the digital age, users are frequently overwhelmed by the sheer volume of text data available across web pages, research articles, and job portals. **PagePilot AI** addresses this challenge as an intelligent, in-browser summarization assistant that transforms dense content into concise, actionable insights—directly within the user’s browsing experience.

The project is developed by a collaborative team consisting of **Ayush Sharma**, **Sumit Kumar Pandit**, and **Harsh**, combining expertise in full-stack development and applied AI systems.

The application follows a modular, scalable architecture divided into three critical layers:

◆ Frontend Layer

A highly responsive and lightweight user interface built using **React.js** and **Tailwind CSS**, seamlessly integrated into the browser via **Chrome Extension Manifest V3**. This layer ensures:

- Real-time content extraction from web pages
- Clean, distraction-free UI for summaries and interactions
- Interactive chat interface for follow-up queries
- Customization options (summary length, tone, language)
- Support for multilingual outputs, including Hinglish

◆ Backend Layer

The backend is powered by a robust **Node.js** environment using **Express.js**, designed for high performance and scalability.

It handles:

- Secure API routing and orchestration
- Data sanitization and preprocessing of scraped content
- **JWT-based authentication** for secure user sessions
- Rate limiting and caching (via Redis) to optimize performance
- Efficient communication between the frontend and AI services

◆ AI Engine Layer

At the core of PagePilot AI lies an advanced AI processing engine powered by the **Google Gemini** and **LangChain**.

This layer is responsible for:

- Context-aware summarization using intelligent text chunking
- Semantic understanding of long-form and complex content
- Conversational Q&A with memory for contextual continuity
- Adaptive responses based on user queries and page context
- High accuracy and relevance in extracted insights

🌟 Impact

PagePilot AI acts as a **real-time knowledge co-pilot**, enabling users to consume information faster, improve productivity, and make better decisions without switching tabs or tools. By embedding intelligence directly into the browser, it redefines how users interact with web content—making the internet more efficient, accessible, and user-centric.

PROBLEM STATEMENT:

Modern internet users face a growing set of challenges when navigating information-dense digital environments, where the sheer scale and complexity of content often hinder efficient consumption and understanding. Time constraints remain one of the most critical issues, as users are expected to sift through lengthy articles, detailed documentation, and extensive job descriptions, making the process not only time-consuming but also cognitively exhausting. This leads to reduced productivity and often causes users to miss key insights hidden within large volumes of text.

In addition to this, token limitations in standard AI models create a significant bottleneck. Many existing solutions fail to process entire web pages due to strict input size constraints, resulting in partial summaries that lack context and completeness. This fragmented understanding reduces the reliability of such tools and limits their practical usability, especially when dealing with research-heavy or multi-section content.

Language barriers further compound the problem, particularly for users who prefer or are more comfortable with hybrid or regional languages such as Hinglish. Most traditional summarization tools are optimized for formal English outputs and do not adapt well to localized linguistic preferences, thereby excluding a large segment of users from fully benefiting from these technologies. This lack of inclusivity restricts accessibility and creates a gap between advanced AI capabilities and real-world user needs.

Another major limitation in current solutions is the static nature of interaction. Most summarization tools operate in a one-directional manner, where users receive a generated summary but are unable to engage further with the content. The absence of an interactive layer prevents users from asking follow-up questions, clarifying doubts, or exploring specific sections in more depth, which ultimately limits comprehension and reduces the overall value of the tool.

Finally, performance and security concerns play a crucial role in user adoption. Many systems struggle with high latency due to repeated API calls, inefficient data handling, and lack of caching mechanisms, resulting in slow response times that disrupt the browsing experience. At the same time, users are increasingly concerned about how their data is processed and stored, highlighting the need for secure architectures that prioritize privacy while maintaining high availability. Ensuring a fast, reliable system with minimal downtime and strong data protection is essential to building trust and delivering a seamless user experience in modern web environments.

OBJECTIVES:

The Crime Prediction System using Machine Learning aims to achieve the following objectives:

1. Develop Real-Time Analysis:

- Enable continuous analysis of across monitored regions.
- Implement data preprocessing algorithms for cleaning and standardizing raw crime records.

2. Implement Predictive Machine Learning Models:

- Create Decision Tree and Random Forest models that predict crime probabilities.
- Reduce prediction error rates through iterative model training and validation.

3. Build Exploratory Data Analysis (EDA) Module:

- Develop visualizations to identify crime hotspots, peak times, and common crime types.
- Implement heatmaps, bar charts, and line graphs for comprehensive data exploration.

4. Design Interactive Dashboard:

- Create an intuitive Streamlit dashboard for users and law enforcement personnel.
- Visualize crime density, prediction results, and trend analytics.

5. Optimize Resource Allocation:

- Provide location-based predictions to help police deploy resources efficiently.
- Enable authorities to prioritize high-risk zones based on model outputs.

6. Reduce Environmental and Social Impact:

- Support preventive policing strategies to reduce overall crime rates.
- Track crime trends to measure the effectiveness of interventions over time.

7. Create Historical Data Repository:

- Store and analyze long-term crime patterns from multiple data sources.
- Generate reports for law enforcement and city planning authorities.

8. Ensure System Scalability:

- Design a modular architecture that can be expanded to include additional crime categories.
- Implement efficient data pipelines for handling city-wide crime datasets.

9. Build Predictive Analytics Pipeline:

- Develop machine learning pipelines to automate model training and evaluation.
- Implement cross-validation and performance metrics for model reliability.

10. Develop User-Friendly Interface:

- Design an accessible dashboard for law enforcement and administrative personnel.
- Enable public access to general crime trend information through web-based tools.

METHODOLOGY / PLANNING OF WORK:

The development of the Crime Prediction System follows a structured methodology that includes data collection, preprocessing, exploratory analysis, model development, evaluation, and deployment. The key aspects of the methodology are as follows:

1. Requirement Analysis

Identifying the key functionalities required for crime data analysis and prediction. Understanding stakeholder needs for data visualization and predictive analytics.

Evaluating technologies for data processing, machine learning, and dashboard development. Gathering requirements from law enforcement agencies to ensure practical implementation.

2. System Design

Architecture Design: Creating a high-level system architecture incorporating data ingestion, preprocessing, machine learning models, and a user interface layer.

User Interface (UI/UX) Design: Designing the dashboard interface using Streamlit for intuitive visualization of crime analytics. Algorithm Design: Developing predictive ML models and EDA modules.

3. Development Tools

Front-End Development:

- Streamlit - Building the interactive web dashboard for crime data visualization.
- Matplotlib / Seaborn - Implementing charts, heatmaps, and statistical plots.

Back-End Development:

- Python - Core programming language for data processing and ML algorithms.
 - Pandas / NumPy - Data manipulation and preprocessing libraries.
 - Scikit-learn - Implementing Decision Tree and Random Forest classifiers.
 - Jupyter Notebook - Interactive development and EDA environment.
 - MySQL / CSV Files - Storing and retrieving crime datasets.
 - Flask (optional) - REST API for model serving.
-

4. Testing

Conducting unit testing for individual data preprocessing and model modules.

Performing integration testing to ensure seamless interaction between the data pipeline and dashboard. Evaluating model performance using metrics such as accuracy, precision, recall, and F1-score.

Validating prediction algorithms using cross-validation on real-world crime datasets.

5. System Architecture

The system follows a layered architecture with data ingestion, preprocessing, analysis, and presentation layers.

Architecture Components:

- Data Ingestion Layer - Interfaces with crime databases and CSV sources to gather historical data.
- Preprocessing Layer - Cleans data by handling missing values, encoding categories, and normalizing features.
- Analysis & ML Layer - Implements EDA visualizations and trains predictive ML models.
- Presentation Layer - Provides interactive dashboards and downloadable reports.

6. UI/UX Design

The application features an intuitive dashboard for law enforcement and city planners. Key UI Components:

- Crime Map - Displays real-time crime density across monitored geographic areas.
- Analytics Dashboard - Shows historical crime patterns, EDA visualizations, and prediction outputs.
- Prediction Panel - Allows users to input parameters and receive crime probability predictions.
- Alert System - Highlights predicted high-risk zones and emerging crime trends.

7. Database Design

The system uses a structured data repository to store crime records, model configurations, and analytics outputs.

Database Tables:

- Crime Records Table: crime_id (Primary Key), location, date_time, crime_type, severity_level
- Prediction Log Table: prediction_id (Primary Key), crime_id (Foreign Key), predicted_crime_type, confidence_score, timestamp
- Region Table: region_id (Primary Key), region_name, coordinates, historical_crime_rate

8. Key Features Implementation

1. Data Preprocessing and Cleaning:

- Removes null and duplicate entries from raw crime datasets.
- Encodes categorical variables such as crime type and location.
- Normalizes numerical features for improved model performance.

2. Exploratory Data Analysis (EDA):

- Generates bar charts to display the most frequent crime types.
- Implements heatmaps to identify geographic crime hotspots.
- Creates time-series line graphs to reveal peak crime hours and seasonal trends.

3. Machine Learning Models:

- Decision Tree: Simple, interpretable model suitable for rule-based crime classification.
 - Random Forest: Ensemble model that improves accuracy by combining multiple decision trees.
 - Both models are evaluated using accuracy, precision, recall, and F1-score metrics.
-

SOURCE CODE SNIPPETS:

data_preprocessing.py

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler

class DataPreprocessor:
    def __init__(self, filepath):
        self.df = pd.read_csv(filepath)
        self.label_encoder = LabelEncoder()
        self.scaler = StandardScaler()

    def clean_data(self):
        # Remove missing values
        self.df.dropna(inplace=True)
        self.df.drop_duplicates(inplace=True)
        # Convert date-time column
        self.df['date'] = pd.to_datetime(self.df['date'])
        self.df['hour'] = self.df['date'].dt.hour
        self.df['month'] = self.df['date'].dt.month
        return self.df

    def encode_features(self):
        self.df['crime_type_enc'] = self.label_encoder.fit_transform(
            self.df['crime_type'])
        self.df['location_enc'] = self.label_encoder.fit_transform(
            self.df['location'])

        return self.df
```

crime_model.py

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

class CrimePredictionModel:
    def __init__(self, model_type='random_forest'):
        if model_type == 'random_forest':
            self.model = RandomForestClassifier(n_estimators=100,
                                                random_state=42)
        else:
            self.model = DecisionTreeClassifier(max_depth=10,
                                                random_state=42)

    def train(self, X, y):
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42)
        self.model.fit(X_train, y_train)
        y_pred = self.model.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        return accuracy, classification_report(y_test, y_pred)

    def predict(self, features):
        return self.model.predict([features])[0]
```

dashboard.py (Streamlit)

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
st.title('Crime Prediction Dashboard')
uploaded_file = st.file_uploader('Upload Crime Data CSV', type='csv')
if uploaded_file:
    df = pd.read_csv(uploaded_file)
    st.subheader('Crime Type Distribution')
    fig, ax = plt.subplots()
    df['crime_type'].value_counts().plot(kind='bar', ax=ax)
    st.pyplot(fig)
```

PURPOSE OF WORK:

The primary purpose of this project is to create an intelligent crime prediction system that enables law enforcement agencies and city authorities to optimize policing strategies and proactively reduce criminal activity. This project is developed to address the critical need for data-driven, responsive crime management solutions that can identify patterns and anticipate future criminal activity.

The Crime Prediction System uses historical crime data analysis to make informed predictions about future crime occurrences, providing significant improvements over traditional reactive law enforcement approaches. By implementing machine learning algorithms for crime classification and prediction, and using Exploratory Data Analysis for pattern identification, the system offers a comprehensive solution for modern urban safety challenges.

This project ultimately aims to reduce crime rates, enhance resource allocation efficiency, lower law enforcement response times, and improve overall quality of life in urban communities through data-driven, proactive crime management.

FUTURE SCOPE:

The Crime Prediction System using Machine Learning has significant potential for future enhancements and improvements.

Some of the key areas for expansion include:

- **Deep Learning Integration:** Implementing advanced neural network models such as LSTM and Transformer architectures for improved time-series crime prediction with greater temporal accuracy.
 - **Real-Time Data Streams:** Incorporating live data feeds from surveillance systems, social media sentiment analysis, and IoT sensors for dynamic, real-time crime prediction updates.
 - **Geographic Information System (GIS) Integration:** Connecting the prediction system with GIS platforms to provide interactive crime hotspot maps and spatial analysis visualizations.
 - **Mobile Application Development:** Creating public-facing mobile applications that provide real-time safety alerts, crime statistics, and suggested safe routes based on model predictions.
 - **Multi-City Scalability:** Expanding the system architecture to handle crime datasets from multiple cities simultaneously, enabling comparative analysis and broader deployment.
 - **Integration with Smart City Platforms:** Connecting the crime prediction system with other smart city initiatives such as traffic management and emergency response systems for coordinated urban safety management.
 - **Explainable AI (XAI):** Implementing model explainability tools to make predictions more transparent and interpretable for law enforcement decision-makers.
-

CONCLUSION:

In conclusion, the challenges of navigating information-heavy digital environments highlight a clear and growing need for smarter, more adaptive tools that go beyond traditional methods of content consumption. PagePilot AI effectively addresses these gaps by integrating intelligent summarization, deep contextual understanding, and interactive AI capabilities into a seamless, in-browser experience. It not only reduces the time and effort required to process large volumes of information but also enhances the quality of understanding by delivering structured, relevant, and concise insights.

By overcoming key limitations such as token constraints, lack of multilingual support, and static one-way interactions, PagePilot AI transforms passive reading into an active, engaging process. Its ability to support conversational queries allows users to explore content more deeply, clarify doubts instantly, and extract precise information tailored to their needs. This shift from static summaries to dynamic interaction significantly improves knowledge retention and decision-making efficiency.

Furthermore, the system's scalable architecture, powered by modern technologies and optimized with caching and secure backend practices, ensures high performance, low latency, and reliable availability. Its support for multilingual outputs, including Hinglish, makes it more inclusive and accessible to a wider audience, bridging the gap between advanced AI systems and real-world usability.

Looking ahead, PagePilot AI has strong potential for further expansion, such as integration with productivity tools, personalized learning systems, and enterprise knowledge platforms. It can evolve into a comprehensive digital assistant that not only summarizes content but also anticipates user intent, recommends relevant information, and automates knowledge workflows.

Ultimately, PagePilot AI represents a step toward a more intelligent and user-centric web ecosystem, where information is not just abundant but also easily digestible, interactive, and meaningful. It empowers users to focus less on navigating information and more on understanding and applying it, thereby enhancing productivity, learning, and overall digital experience.

REFERENCES:

- Python Documentation - Reference for building Python-based data science applications. <https://docs.python.org/3/>
 - Scikit-learn Documentation - Machine learning library for Python. <https://scikit-learn.org/stable/>
 - Streamlit Documentation - Framework for building interactive data dashboards. <https://docs.streamlit.io/>
 - Pandas Documentation - Data manipulation and analysis library. <https://pandas.pydata.org/docs/>
 - Perry, W.L. et al. (2013) - 'Predictive Policing: The Role of Crime Forecasting in Law Enforcement Operations', RAND Corporation.
 - Crime Prediction Using Machine Learning: A Survey - International Journal of Computer Applications, 2022
 - Random Forest Algorithm for Crime Prediction and Analysis - IEEE Smart Cities Conference Proceedings, 2023
 - Exploratory Data Analysis Techniques for Crime Pattern Recognition - Data Science Journal, 2023
-